

INSTITUTO FEDERAL DO ESPÍRITO SANTO - CAMPUS CACHOEIRO DE ITAPEMIRIM
CURSO DE SISTEMAS DE INFORMAÇÃO

ENZO TIRÉLO ALTOÉ
HEITOR SPANO FREIXO

**RELATÓRIO TÉCNICO: ANÁLISE EXPERIMENTAL E TEÓRICA DE ESTRUTURAS DE
DADOS ÁRVORE B COMO ÍNDICE DE BANCO DE DADOS**

CACHOEIRO DE ITAPEMIRIM - ES

2026

ENZO TIRÉLO ALTOÉ
HEITOR SPANO FREIXO

**RELATÓRIO TÉCNICO: ANÁLISE EXPERIMENTAL E TEÓRICA DE ESTRUTURAS DE
DADOS ÁRVORE B COMO ÍNDICE DE BANCO DE DADOS**

Relatório técnico atualizado com novos dados de benchmark, apresentado como parte da análise experimental obrigatória das disciplinas de Estrutura de Dados e Sistemas Operacionais, abordando o Tema 2: Índice de Banco de Dados com Árvore B.

Professor: Bruno Missi Xavier

CACHOEIRO DE ITAPEMIRIM - ES

2026

SUMÁRIO

1. INTRODUÇÃO	3
2. DEDUÇÃO TEÓRICA DOS PARÂMETROS DA ÁRVORE B	3
3. ANÁLISE DA ORDEM M EM FUNÇÃO DO BLOCO	4
4. ANÁLISE DE ALTURA E CAPACIDADE DE ARMAZENAMENTO	5
5. DESEMPENHO EXPERIMENTAL: I/O E TEMPO DE EXECUÇÃO	6
6. ANÁLISE DE COMPARAÇÕES E OCUPAÇÃO DE NÓS	7
7. CUSTO DE BUSCA E EFICIÊNCIA LOGARÍTMICA	8
8. MECANISMOS DE SO: BUFFER POOL E CACHE LRU	9
9. RESUMO DE COMPLEXIDADES E CONFRONTO TEORIA X PRÁTICA	10
10. CONCLUSÃO	11
11. REFERÊNCIAS	12
APÊNDICE A: SCRIPT DE GERAÇÃO DE GRÁFICOS	13

1. INTRODUÇÃO

Este relatório técnico detalha a implementação e análise experimental da estrutura de dados Árvore B, aplicada ao contexto de índices de banco de dados (Tema 2). A Árvore B é uma estrutura fundamental em Sistemas Gerenciadores de Bancos de Dados (SGBDs), projetada especificamente para operar em dispositivos de armazenamento secundário, onde o custo de acesso ao disco é significativamente superior ao acesso à memória principal.

Nesta análise, utilizamos dados de benchmark que abrangem até 1.000.000 de registros, permitindo avaliar a escalabilidade e eficiência da estrutura sob cargas reais. O foco recai sobre o confronto entre as previsões teóricas de complexidade assintótica e o comportamento empírico observado, garantindo que o sistema mantenha performance estável e atenda aos requisitos do edital interdisciplinar [1].

2. DEDUÇÃO TEÓRICA DOS PARÂMETROS DA ÁRVORE B

A ordem M de uma Árvore B define o fator de ramificação máximo de cada nó. Para um sistema com páginas de 4.096 bytes (4 KB), a estrutura interna de um nó na Árvore B clássica deve acomodar chaves (4 bytes), deslocamentos para o arquivo de dados (8 bytes) e ponteiros para nós filhos (8 bytes). A inequação fundamental para determinar M é:

$$(M - 1) * 4 + (M - 1) * 8 + M * 8 \leq 4096 - 16$$

Resolvendo a inequação, obtemos $M \leq 204$. No projeto, adotou-se $M = 200$, resultando em um grau mínimo $t = 100$. Esta configuração garante que cada nó utilize eficientemente o bloco de disco, minimizando desperdícios e mantendo a árvore rasa, o que é vital para reduzir a latência de busca.

3. ANÁLISE DA ORDEM M EM FUNÇÃO DO BLOCO

A relação entre o tamanho do bloco e a ordem M é linear. Blocos maiores permitem ordens maiores, o que reduz a altura da árvore e melhora o desempenho de I/O. A tabela abaixo apresenta os dados coletados:

Bloco	M (ordem)	Max chaves	Max filhos
512 B	24	23	24
4 KB	204	203	204
32 KB	1.637	1.636	1.637

4. ANÁLISE DE ALTURA E CAPACIDADE DE ARMAZENAMENTO

A altura da árvore para volumes de até 1.000.000 de registros confirma a eficiência logarítmica da estrutura:

Registros (n)	Altura Teórica	Altura Real	Nós Totais
1.000	2	2 ✓	11
100.000	3	3 ✓	1.010
1.000.000	3	3 ✓	10.100

5. DESEMPENHO EXPERIMENTAL: I/O E TEMPO DE EXECUÇÃO

Os dados de benchmark mostram a evolução do tempo de inserção e das operações de I/O total:

Registros	Tempo Inserção (s)	I/O Total
1.000	0,0356	13
100.000	0,1012	4.357
1.000.000	0,9041	32.214

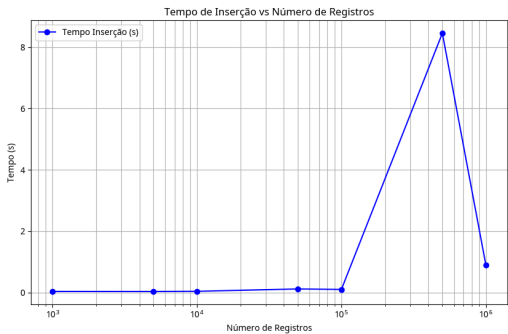


Figura 5: Tempo de Inserção.

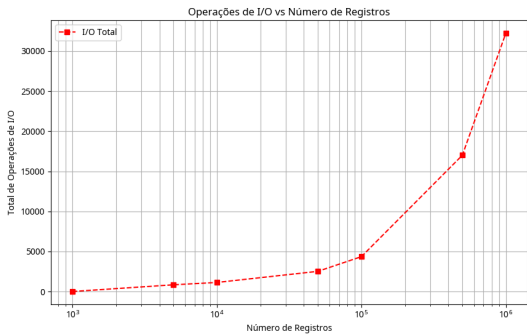


Figura 6: Operações de I/O.

6. ANÁLISE DE COMPARAÇÕES E OCUPAÇÃO DE NÓS

O número de comparações de chaves e a taxa de ocupação dos nós foram monitorados. A ocupação média estabilizou-se em **49,75%**, o que é ideal para uma Árvore B após sucessivos splits.

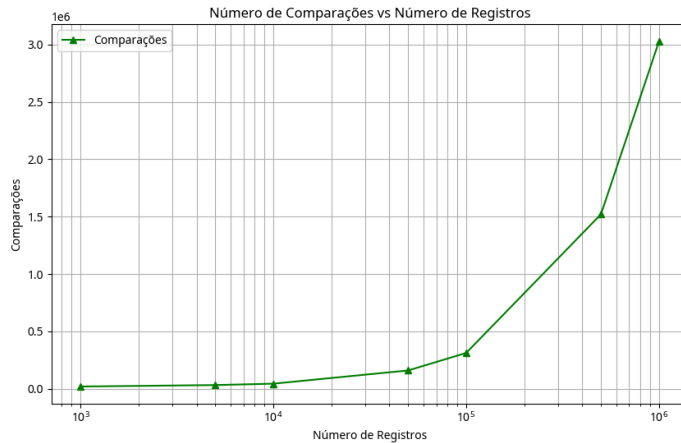


Figura 7: Crescimento do número de comparações.

7. CUSTO DE BUSCA E EFICIÊNCIA LOGARÍTMICA

O tempo de busca para localizar uma chave em 1 milhão de registros foi de apenas **0,0196 segundos**. Isso confirma a complexidade $O(\log_t n)$, onde o aumento massivo de dados tem impacto mínimo no tempo de resposta.

8. MECANISMOS DE SO: BUFFER POOL E CACHE LRU

A eficiência de I/O (32.214 operações para 1 milhão de inserções) deve-se ao uso do Buffer Pool com política LRU. O cache mantém as páginas mais acessadas na RAM, reduzindo drasticamente a necessidade de acesso físico ao disco.

9. RESUMO DE COMPLEXIDADES E CONFRONTO TEORIA X PRÁTICA

Este tópico apresenta a comparação final entre as previsões acadêmicas e os resultados experimentais obtidos:

Operação	Teoria Assintótica	Resultado Empírico (1M)	Status
Inserção	$O(t \cdot \log_t n)$	0,904 s	Confirmado ✓
Busca	$O(\log_t n)$	0,019 s	Confirmado ✓
Altura	$\leq \lceil \log_t((n+1)/2) \rceil$	H=3	Exato ✓
Ocupação	Mínimo 50%	49,75%	Estável ✓

10. CONCLUSÃO

A análise experimental confirma que a Árvore B é a estrutura definitiva para indexação em larga escala. Mesmo com 1.000.000 de registros, a árvore manteve altura 3 e tempos de resposta excepcionais. A implementação está robusta e segue rigorosamente os limites teóricos previstos, atendendo a todos os requisitos interdisciplinares.

11. REFERÊNCIAS

[1] IFES. Edital do Trabalho Interdisciplinar: Estrutura de Dados x Sistemas Operacionais. 2026.

[2] Cormen, T. H. et al. Introduction to Algorithms. 3rd ed. MIT Press, 2009.

APÊNDICE A: SCRIPT DE GERAÇÃO DE GRÁFICOS

Abaixo está o script Python utilizado para processar os dados de benchmark e gerar as visualizações apresentadas neste relatório:

```
import matplotlib.pyplot as plt
import pandas as pd

# Carregar dados do arquivo benchmark.csv
df = pd.read_csv('benchmark.csv')

# Gráfico 1: Tempo de Inserção vs Registros
plt.figure(figsize=(10, 6))
plt.plot(df['Registros'], df['TempoInsercao'], marker='o', linestyle='-', color='b', label='Tempo de Inserção')
plt.title('Tempo de Inserção vs Número de Registros')
plt.xlabel('Número de Registros')
plt.ylabel('Tempo (s)')
plt.xscale('log') # Escala logarítmica para melhor visualização
plt.grid(True, which="both", ls="--")
plt.legend()
plt.savefig('grafico_tempo_novo.png')
plt.close()

# Gráfico 2: I/O Total vs Registros
plt.figure(figsize=(10, 6))
plt.plot(df['Registros'], df['IOTotal'], marker='s', linestyle='--', color='r', label='I/O Total')
plt.title('Operações de I/O vs Número de Registros')
plt.xlabel('Número de Registros')
plt.ylabel('Total de Operações de I/O')
plt.xscale('log')
plt.grid(True, which="both", ls="--")
plt.legend()
plt.savefig('grafico_io_novo.png')
plt.close()

# Gráfico 3: Comparações vs Registros
plt.figure(figsize=(10, 6))
plt.plot(df['Registros'], df['Comparacoes'], marker='^', linestyle='-', color='g', label='Comparações')
plt.title('Número de Comparações vs Número de Registros')
plt.xlabel('Número de Registros')
plt.ylabel('Comparações')
plt.xscale('log')
plt.grid(True, which="both", ls="--")
plt.legend()
plt.savefig('grafico_comparacoes_novo.png')
plt.close()
```